

**BEATRIZ DOS SANTOS
BRENDA RIBEIRO ALVES
MARIA EDUARDA BEZERRA**

PROJETO INTEGRADOR - 3º TERMO

Manutenção Industrial 4.0

LIMEIRA – SP

2026

Sumário

1. LEVANTAMENTO DE REQUISITOS.....	1
1.1. REQUISITOS FUNCIONAIS	1
1.2. REQUISITOS NÃO FUNCIONAIS	1
2. PRIORIZAÇÃO.....	2
3. FIGMA.....	4
4. METODOLOGIAS ÁGEIS.....	5
4.1. METODOLOGIA KANBAN	5
5. README E VERSIONAMENTO (GIT)	6
6. DIAGRAMA.....	7
6.1. DIAGRAMA DE CLASSE	7
6.2. DIAGRAMA DE SEQUÊNCIA.....	8
6.3. DIAGRAMA BANCO DE DADOS.....	9
7. SOBRE O SISTEMA	10
7.1. VISÃO GERAL.....	10
7.2. TECNOLOGIAS UTILIZADAS.....	10
7.3. ENTIDADES PRINCIPAIS.....	10
7.4. FLUXO PRINCIPAL.....	10
7.5. FLUXO DE PREVENTIVA	11
7.6. INDICADORES DE MANUTENÇÃO.....	11
7.7. USUÁRIOS E PERMISSÕES.....	11
7.8. ESTADO ATUAL DO PROJETO	11
8. CONCLUSÃO	12

MANUTENÇÃO INDUSTRIAL

Situação Problema: "Na era da Indústria 4.0, a disponibilidade das máquinas é crucial para a produtividade. Paradas não planejadas custam milhares de reais e atrasam toda a cadeia produtiva. Fábricas modernas buscam migrar de anotações em papel para sistemas digitais que prevejam e registrem manutenções de forma eficiente." "O Senai contratou sua consultoria para acabar com as 'fichas de papel' penduradas nas máquinas. Eles precisam de um sistema centralizado onde a equipe de manutenção possa registrar intervenções e os gerentes possam acompanhar a saúde dos equipamentos em tempo real."

Desafio: "Sua equipe deve implementar o Back-End do 'MaintSys', uma API para controle de manutenção de maquinário industrial. O sistema deve garantir a integridade dos dados e permitir que técnicos registrem atividades através de futuros tablets ou terminais industriais."

Funcionalidades Essenciais:

1. Gestão de Técnicos e Acesso: Cadastro de técnicos especializados com autenticação segura.
2. Inventário de Máquinas: Cadastro de equipamentos (Número de Série, Modelo, Localização no Galpão, Data de Instalação).
3. Ordens de Serviço (O.S.): Criação de ordens de manutenção (Preventiva ou Corretiva), vinculando um técnico a uma máquina específica.
4. Histórico de Manutenções: Log completo de todas as intervenções realizadas em uma máquina, permitindo análise de reincidência de defeitos.
5. Alertas de Status: Sistema para notificar quando uma máquina muda de status (ex: "Máquina 04 em Parada Crítica" ou "Manutenção Concluída").

Requisitos e Restrições:

- Back-End desenvolvido em Laravel (PHP).
- Banco de dados relacional MySQL.
- Utilizar o Eloquent ORM para modelar as tabelas e relações (ex: Uma Máquina tem muitas Ordens de Serviço).
- API RESTful retornando JSON.
- Código seguindo padrões PSR e Clean Code.
- Publicação no GitHub e testes via Insomnia/Postman.

1. LEVANTAMENTO DE REQUISITOS

O levantamento de requisitos é o processo de identificar, analisar e documentar as necessidades reais dos clientes e usuários.

1.1. REQUISITOS FUNCIONAIS

Especificam as interações e os serviços que o sistema deve oferecer para atender às necessidades do usuário.

RF1 O sistema deve coletar dados de sensores (vibração, temperatura e pressão) em tempo real.
RF2 O sistema deve armazenar os dados coletados em banco de dados histórico.
RF3 O sistema deve analisar automaticamente os dados utilizando algoritmos preditivos.
RF4 O sistema deve gerar alertas automáticos quando detectar anomalias.
RF5 O sistema deve permitir o cadastro de máquinas e equipamentos.
RF6 O sistema deve permitir o cadastro de usuários com diferentes níveis de acesso.
RF7 O sistema deve gerar relatórios de desempenho e histórico de falhas.
RF8 O sistema deve enviar notificações por e-mail ou aplicativo móvel. (Versão - Mobile)
RF9 O sistema deve permitir agendamento de manutenções preventivas.
RF10 O sistema deve exibir dashboard com indicadores (MTBF, MTTR, disponibilidade). (Versão - Mobile)

1.2. REQUISITOS NÃO FUNCIONAIS

Estabelecem os critérios que restringem ou qualificam a operação do sistema, garantindo sua viabilidade e experiência.

RNF1 Desempenho: O sistema deve processar e exibir dados em até 3 segundos após a coleta.
RNF2 Disponibilidade: O sistema deve estar disponível 99% do tempo.
RNF3 Segurança: O sistema deve possuir autenticação por login e senha criptografada.
RNF4 Escalabilidade: O sistema deve suportar no mínimo 500 equipamentos cadastrados.
RNF5 Usabilidade: A interface deve ser intuitiva e permitir treinamento em no máximo 4 horas.
RNF6 Confiabilidade: Os dados devem possuir backup automático diário.
RNF7 Compatibilidade: O sistema deve funcionar em navegadores atualizados (Chrome, Edge e Firefox).
RNF8 Manutenibilidade: O sistema deve permitir atualização de software sem interromper o funcionamento por mais de 10 minutos.

2. PRIORIZAÇÃO

O método MoSCoW é uma técnica de priorização que organiza requisitos em quatro níveis. Ele serve para gerenciar o foco e alinhar expectativas com os stakeholders, evitando o desperdício de recursos enquanto o projeto ainda não está pronto.

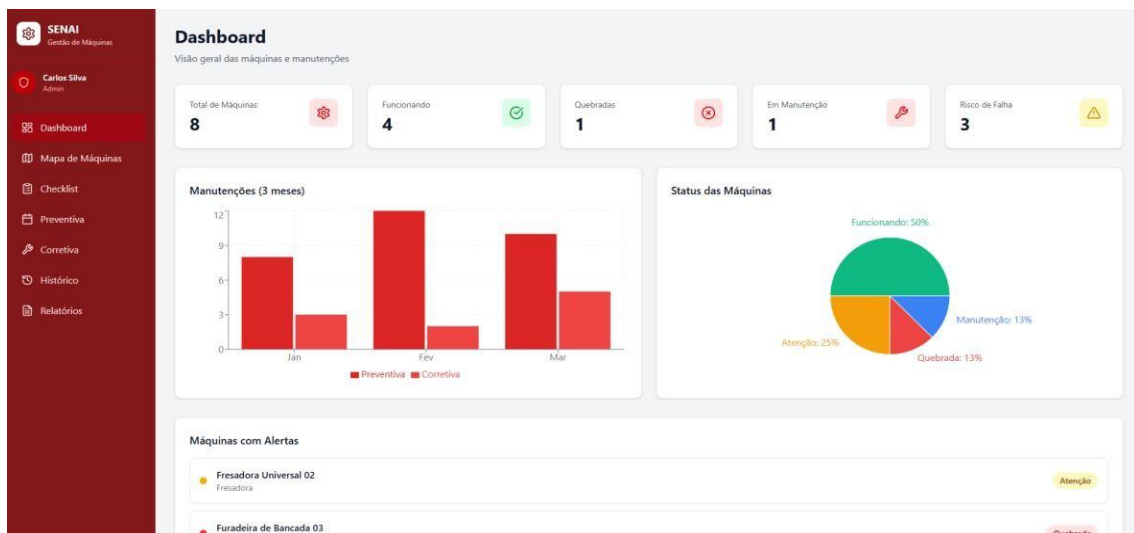
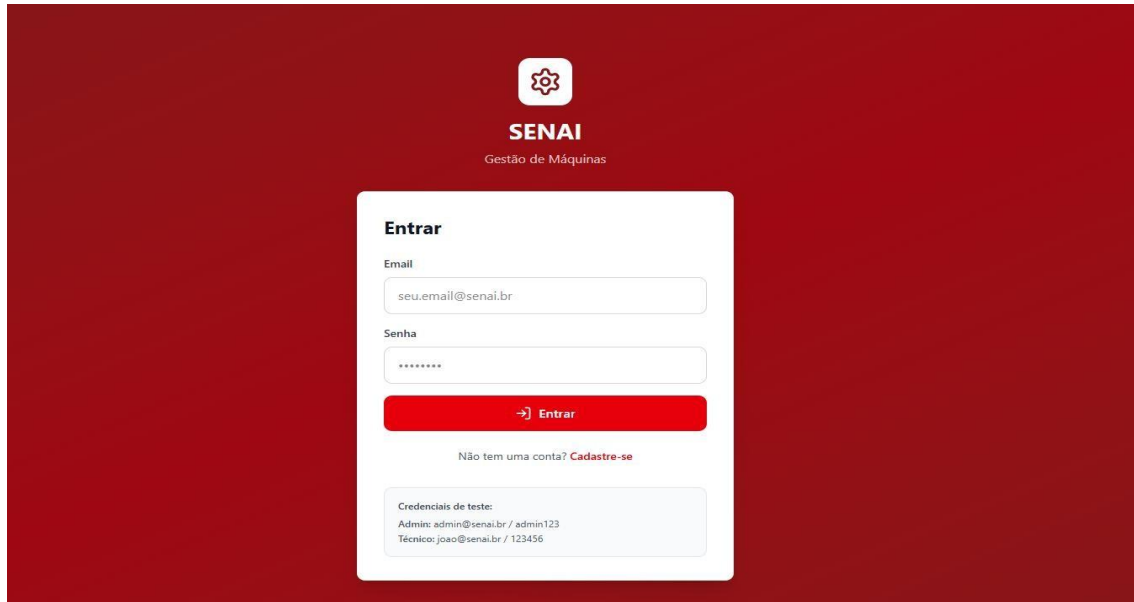
- **MUST HAVE (Deve ter):** Crítico. Sem isso, o projeto não é entregue ou não funciona. Essencial.
- **SHOULD HAVE (Deveria ter):** Importante, mas não vital. Pode ser entregue logo após o essencial. Alta Prioridade.
- **COULD HAVE (Poderia ter):** Desejável. É um diferencial que só será feito se sobrar tempo/recurso. Opcional.
- **WON'T HAVE (Não terá):** Fora de escopo para este momento. Ajuda a focar no que importa agora. Futuro.

Requisito	Priorização	Explicação
RF1 – Coletar dados de sensores em tempo real	M. H	Essencial para o funcionamento do sistema de monitoramento preditivo.
RF2 – Armazenar dados em banco histórico	M. H	Necessário para análises, rastreabilidade e geração de relatórios.
RF3 – Análise automática com algoritmos preditivos	M. H	Principal diferencial do sistema de manutenção preditiva.
RF4 – Gerar alertas automáticos de anomalias	M. H	Fundamental para evitar falhas críticas e reduzir tempo de parada.
RF5 – Cadastro de máquinas e equipamentos	M. H	Base estrutural para organização e monitoramento dos ativos.
RF6 – Cadastro de usuários com níveis de acesso	M. H	Essencial para segurança e controle de permissões.
RF7 – Gerar relatórios de desempenho e falhas	S. H	Importante para análise estratégica e tomada de decisão.
RF8 – Enviar notificações por e-mail/app	S. H	Agrega agilidade na comunicação de eventos críticos.
RF9 – Agendamento de manutenções preventivas	S. H	Complementa a estratégia de manutenção preditiva.
RF10 – Dashboard com indicadores (MTBF, MTTR, disponibilidade)	S. H	Importante para visualização gerencial e acompanhamento de desempenho.

RNF1 – Processamento em até 3 segundos	M. H	Requisito crítico de desempenho para garantir resposta em tempo real.
RNF2 – Disponibilidade de 99%	M. H	Garante confiabilidade mínima para operação industrial.
RNF3 – Autenticação com senha criptografada	M. H	Requisito essencial de segurança da informação.
RNF4 – Suportar 500 equipamentos	S. H	Importante para escalabilidade inicial do sistema.
RNF5 – Interface intuitiva (treinamento até 4h)	S. H	Melhora adoção e reduz custos de capacitação.
RNF6 – Backup automático diário	M. H	Essencial para garantir integridade e recuperação de dados.
RNF7 – Compatível com Chrome, Edge e Firefox	M. H	Importante para acessibilidade, mas não crítico na fase inicial.
RNF8 – Atualização com interrupção máxima de 10 min	C. H	Desejável para continuidade operacional, mas pode ser evoluído posteriormente.

3. FIGMA

Foi a peça-chave para validar a experiência do usuário (UX) e a interface (UI) antes do código. O protótipo de alta fidelidade no Figma garantiu o alinhamento visual com a marca SENAI, eliminou erros de usabilidade e acelerou o desenvolvimento ao servir como guia exato para a implementação técnica.



Você pode visualizar o layout completo através deste link: [FIGMA](#)

4. METODOLOGIAS ÁGEIS

São formas de gerenciar projetos focadas em entregas rápidas, adaptabilidade a mudanças e trabalho em equipe constante.

4.1. METODOLOGIA KANBAN

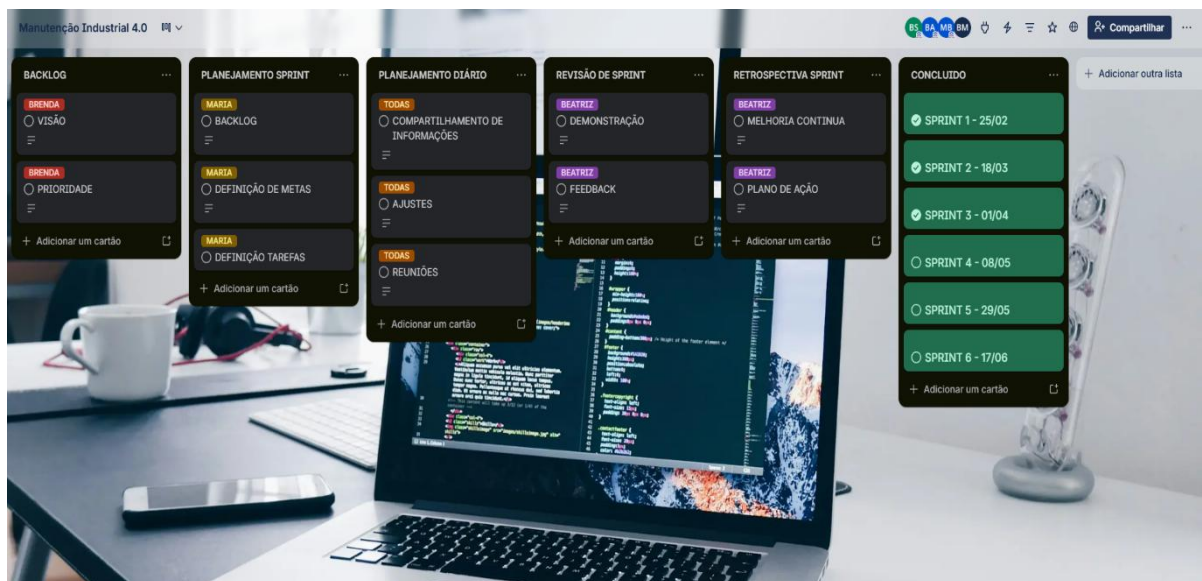
O Kanban é um método de gestão visual focado na eficiência do fluxo de trabalho.

Utilizando cartões e colunas (como "A Fazer", "Em Execução" e "Feito"), ele permite que a equipe visualize o status de cada tarefa em tempo real.

O grande diferencial é a aplicação de limites de WIP (Work In Progress), que impedem a sobrecarga da equipe e garantem que o foco esteja na conclusão, não apenas no início de novas demandas.

Vantagens: Alta flexibilidade para mudanças de prioridade e transparência total do processo, facilitando a identificação imediata de gargalos.

Desvantagens: A ausência de prazos rígidos (sprints) pode dificultar a estimativa de grandes projetos a longo prazo. Além disso, exige alta disciplina da equipe para que o quadro reflita a realidade.



Clique [Aqui](#) para abrir o documento original

5. README E VERSIONAMENTO (GIT)

No desenvolvimento do sistema para o Senai, a transição do físico para o digital exige clareza e segurança. Para garantir que o projeto seja sustentável e profissional, utilizamos duas ferramentas fundamentais: o arquivo README e o Versionamento (Git).

README

Manutenção Industrial 4.0

Status do Projeto: 🟡 Em desenvolvimento

Equipe
Beatriz dos Santos
Brenda Ribeiro Alves
Maria Eduarda Bezerra

🎯 Objetivo

Este projeto visa a digitalização completa do fluxo de manutenção fabril, substituindo registros físicos por uma inteligência de dados centralizada.

🔗 Links do Projeto

- [Protótipo no Figma](#)
- [Documentação Word](#)
- [Gestão no Trello](#)
- [Notion](#)

🌐 Funcionalidades e Tecnologias

- Definição do stack (Laravel/MySQL) e das funcionalidades core.
- Garante que o sistema seja robusto, padronizado e fácil de manter.

✅ Levantamento Requisitos (Funcionais e Não Funcionais)

- Mapeamento do que o sistema faz e seus parâmetros de performance/segurança.
- Alinha a entrega técnica com a necessidade real do cliente.

🚀 Priorização (MoSCoW)

- Filtro que separa o que é vital (Must Have) do que é opcional.
- Garante foco total na entrega do MVP (valor imediato).

📋 Metodologia Kanban

- Gestão visual do fluxo de trabalho e das tarefas.
- Evita sobrecarga da equipe e dá transparência ao progresso do projeto.

🎨 Prototipagem

- Desenho e modelagem inicial da solução.
- Valida a ideia com o usuário final antes do investimento em programação.

🔄 Versionamento (Git)

- Histórico e controle de todas as alterações no código-fonte.
- Protege contra erros e permite a colaboração segura entre desenvolvedores.

📄 Documentação

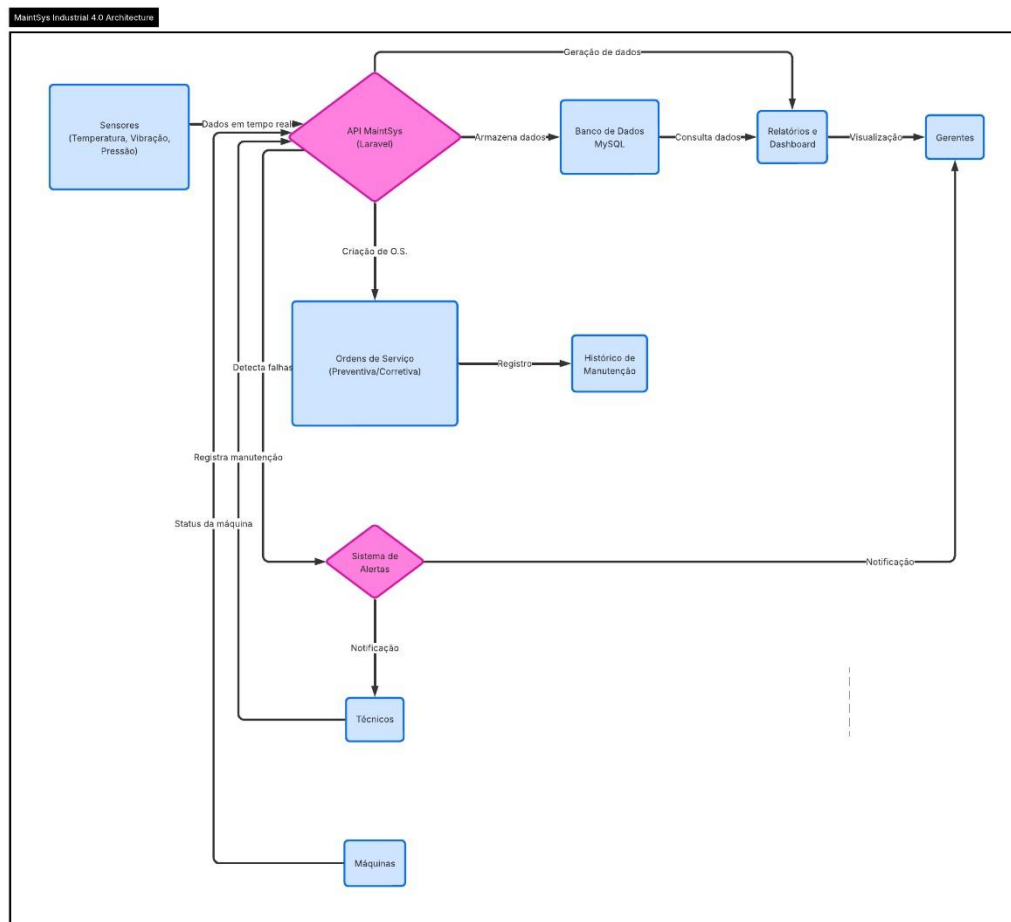
- Registro técnico e funcional escrito do projeto.
- Preserva o conhecimento e facilita futuras expansões ou manutenções.

6. DIAGRAMA

Um diagrama é uma representação gráfica e esquemática de informações. Ele utiliza simbologia padronizada para descrever estruturas, fluxos de trabalho ou interações entre componentes de um sistema.

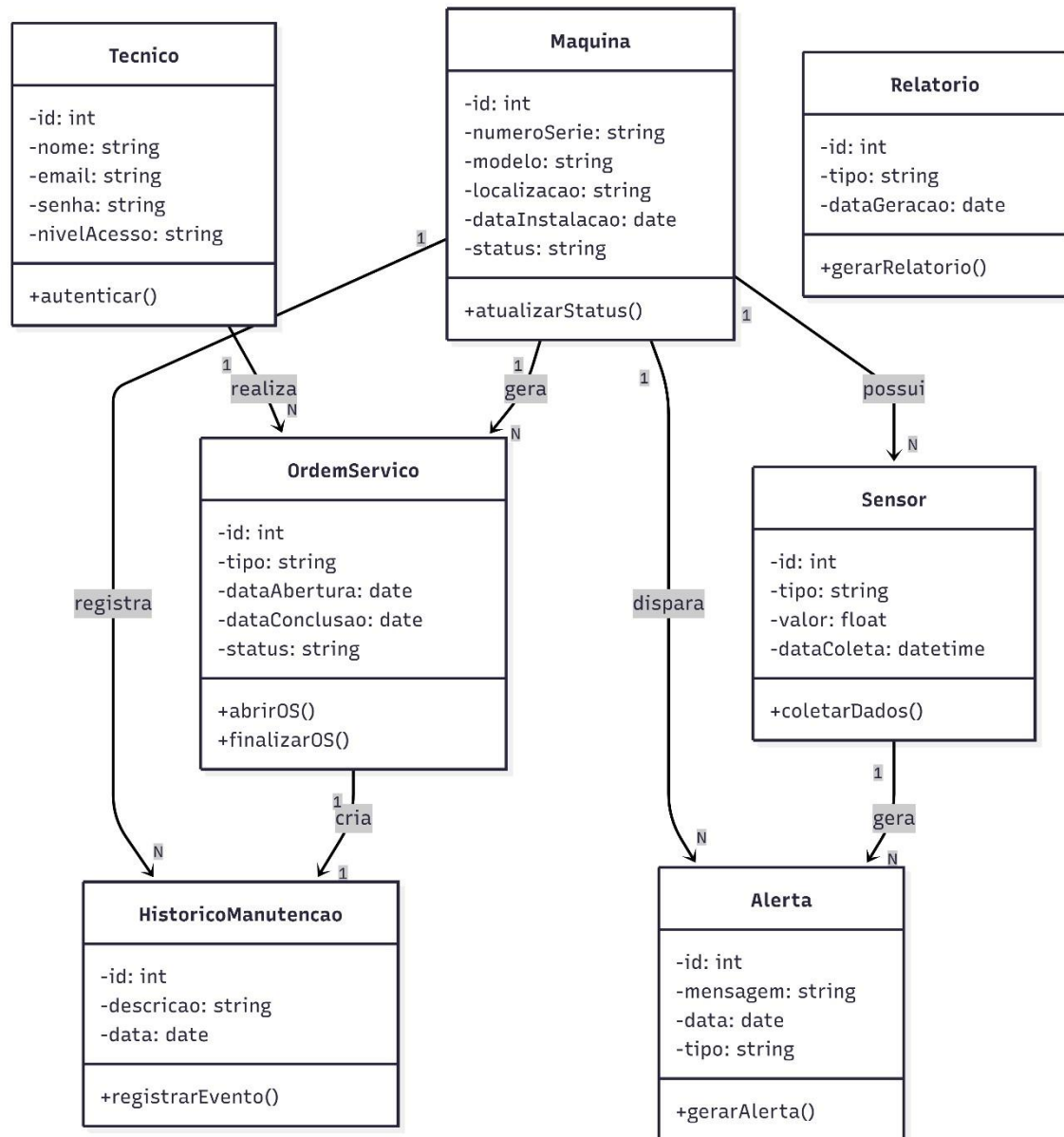
6.1. DIAGRAMA DE CLASSE

Este diagrama representa o ecossistema da solução, ilustrando o fluxo de dados desde a captura física nos sensores até a tomada de decisão gerencial.



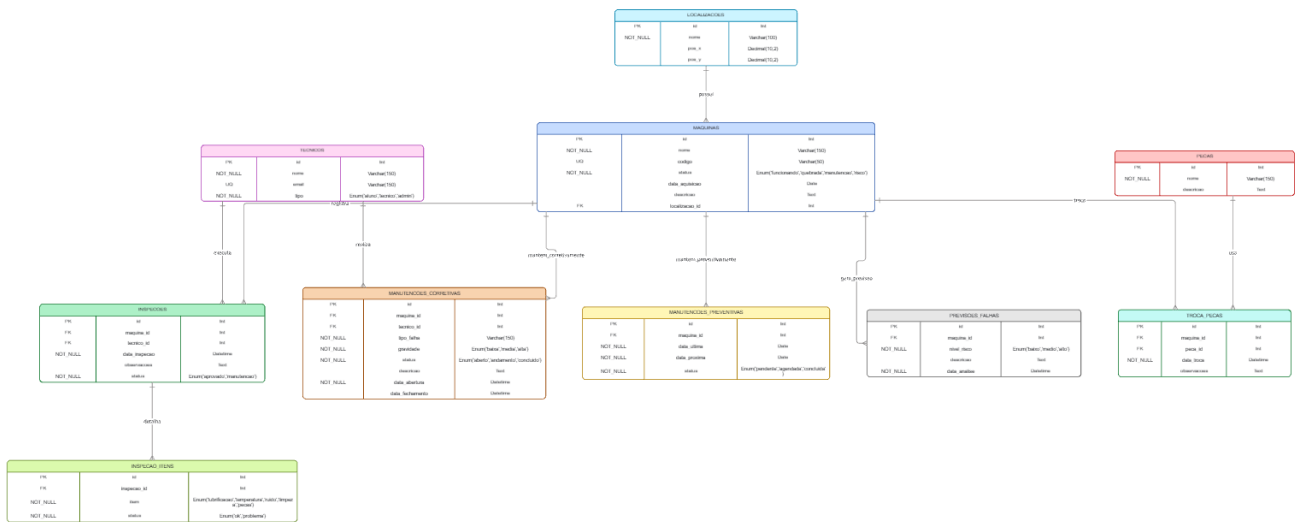
6.2. DIAGRAMA DE SEQUÊNCIA

O diagrama detalha a estrutura interna do sistema, definindo as propriedades e responsabilidades de cada objeto dentro da aplicação Laravel.



6.3. DIAGRAMA BANCO DE DADOS

Serve para visualizar, planejar e documentar a estrutura de dados de um sistema, mapeando tabelas, colunas e seus relacionamentos.



Para conferir o detalhamento desse diagrama, acesse o link: [MIRO](#)

7. SOBRE O SISTEMA

O MaintSys é uma aplicação desenvolvida com Laravel 12 + Filament 3 para gerenciamento de manutenção industrial. O sistema centraliza o cadastro de setores, máquinas, ocorrências, inspeções, manutenções corretivas e preventivas, checklists técnicos, peças trocadas e histórico operacional.

7.1. VISÃO GERAL

O objetivo do MaintSys é acompanhar todo o ciclo de vida da manutenção industrial, desde o cadastro da infraestrutura até a execução e acompanhamento das manutenções.

7.2. TECNOLOGIAS UTILIZADAS

- Laravel 12
- Filament 3
- Livewire
- Spatie Laravel Permission
- MySQL

7.3. ENTIDADES PRINCIPAIS

Setor: Representa áreas da empresa como Usinagem, CNC e Solda.

Máquina: Equipamentos industriais com informações técnicas e preventivas.

Ocorrência: Chamados de manutenção abertos por falhas.

Inspeção: Diagnóstico técnico da ocorrência.

Corretiva: Manutenção executada para solucionar problemas.

Preventiva: Manutenção planejada baseada em periodicidade.

Checklist: Estrutura flexível para inspeções e manutenções.

Histórico: Registro de eventos e ações do sistema.

7.4. FLUXO PRINCIPAL

1. Cadastro de setores.
2. Cadastro de máquinas.

3. Configuração de preventivas.
4. Abertura de ocorrências.
5. Inspeção técnica.
6. Execução de corretiva.
7. Finalização da manutenção.
8. Atualização automática do histórico.

7.5. FLUXO DE PREVENTIVA

O sistema calcula automaticamente a próxima preventiva com base na última manutenção realizada. Quando uma preventiva é finalizada, uma nova preventiva futura pode ser criada automaticamente.

7.6. INDICADORES DE MANUTENÇÃO

Disponibilidade: Calculada com base em MTBF e MTTR.

MTBF: Tempo médio entre falhas.

MTTR: Tempo médio de reparo.

7.7. USUÁRIOS E PERMISSÕES

O sistema utiliza controle de acesso baseado em roles:

- admin
- tecnico
- supervisor

7.8. ESTADO ATUAL DO PROJETO

As áreas mais completas do sistema atualmente são:

- Máquinas
- Ocorrências
- Checklist Modelo

Algumas resources ainda estão em desenvolvimento, principalmente as relacionadas a inspeções, corretivas e preventivas.

8. CONCLUSÃO

O MaintSys consolida-se como uma solução robusta para a gestão de manutenção industrial no contexto da Indústria 4.0. A plataforma integra controle operacional, gestão preventiva, fluxo de manutenção corretiva, checklists técnicos e indicadores de desempenho (KPIs), mantendo um histórico completo de todas as atividades. O desenvolvimento deste projeto viabilizou a transição crítica e necessária dos registros físicos para um ecossistema digital moderno, centralizado e seguro.

O alicerce dessa jornada foi a criação pioneira da versão Web. Essa etapa inicial estabeleceu a infraestrutura e a lógica de negócios indispensáveis para viabilizar, posteriormente, a expansão e a implementação da versão mobile.

O sucesso do projeto sustentou-se em três pilares fundamentais:

Evolução Estratégica: A partir da base Web estável, o sistema expandiu-se para o ambiente mobile utilizando a tecnologia FlutterFlow. A aplicação rigorosa da metodologia MoSCoW foi decisiva para priorizar os requisitos de maior valor estratégico, garantindo entregas ágeis e focadas.

Gestão e Arquitetura de Dados: Implementou-se um controle estrito sobre as ordens de serviço, alocação de técnicos e históricos de intervenção. A modelagem de dados foi concebida por meio de diagramas verticais que se converteram diretamente na estrutura do banco de dados. Essa definição precisa e a tipagem correta dos dados logo na fase inicial eliminaram retrabalhos na criação dos schemas do backend.

Usabilidade e Agilidade: A interface, projetada com foco na experiência do usuário, foi inicialmente prototipada no Figma e transposta com agilidade para o ambiente mobile, otimizando o tempo de desenvolvimento. O fluxo de trabalho foi gerenciado via metodologia Kanban, com o quadro organizado por camadas funcionais — uma estratégia que impediu que pendências visuais ou de design bloqueassem o avanço da lógica do sistema.

Diante dessa trajetória de desenvolvimento, o MaintSys cumpre com excelência o desafio proposto. Ao elevar o padrão dos processos de manutenção do Senai, o projeto entrega uma plataforma escalável, madura, de alta performance e estritamente orientada a dados.